Institut Teknologi Bandung

Directorate of Continuing Professional Education

in partnership with

CHAPTER 18

Best Practices for the Real World

Past "works okay": automatic hyperparameter search, ensembling, distributed training across devices, and a near-free 2× from lower-precision arithmetic.

Duration 3 hours (2 sessions)

Instructor Rahman Indra Kesuma, S.Kom., M.Cs.

Source Chollet & Watson, Deep Learning with Python 3e — chapter 18

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Distinguish **hyperparameters from parameters**, and say why hyperparameter search cannot use gradient descent.
2. Define a **search space** with KerasTuner and run a Bayesian optimization search, then retrain the best configurations properly.
3. Explain **validation-set overfitting** in tuning, and how to avoid trusting a contaminated number.
4. Ensemble models with a **weighted average**, and explain why diversity matters more than individual quality.
5. Choose between **data parallelism and model parallelism**, and say what each one is actually for.
6. Use the **DeviceMesh and LayoutMap** APIs to shard a model across devices under the JAX backend.
7. Explain **floating-point precision**, and use float16 inference, mixed-precision training, and loss scaling correctly.
8. Apply **int8 quantization** to a trained model, and explain why the scale-and-unscale trick preserves the result.

BOOK
Chapter 18 — full text

NOTEBOOK
01_kerastuner_search.ipynb

NOTEBOOK
02_ensembling_and_weights.ipynb

NOTEBOOK
03_data_and_model_parallel_jax.ipynb

NOTEBOOK
04_mixed_precision_and_loss_scaling.ipynb

NOTEBOOK
05_int8_quantization.ipynb

REPO
Author's official notebook

01

Getting the most out of your models

From "works okay" to "wins machine learning competitions".

SECTION 18.1.1

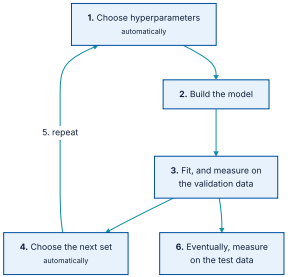
Hyperparameters are the decisions backpropagation cannot make

These architecture-level choices are called **hyperparameters**, to distinguish them from the model's **parameters**, which are trained by backpropagation.

> Experienced engineers build intuition about what works. But **there are no formal rules**, and initial decisions are almost always suboptimal even with very good intuition. **It should not be your job to fiddle with hyperparameters all day.**

SECTION 18.1.1

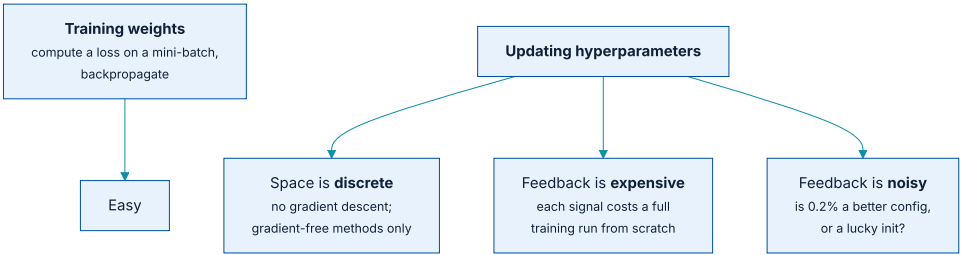
The optimization loop



Automatic hyperparameter optimization is an entire field of research, and an important one.

The key to the whole process is **step 4** — the algorithm that analyses the relationship between validation performance and hyperparameter values to choose what to try next. Bayesian optimization, genetic algorithms, plain random search.

Three reasons this is not like training weights



Training weights is easy by comparison: one loss, one backward pass.

The third point is the one people underestimate. If a run performs 0.2% better, was that **a better configuration** or **a lucky weight initialization**? Without answering that, you are optimizing noise.

KerasTuner: replace a constant with a range

listing 18.1 PYTHON

```python
import keras
from keras import layers

def build_model(hp):
    units = hp.Int(name="units", min_value=16, max_value=64, step=16)
    model = keras.Sequential(
        [
            layers.Dense(units, activation="relu"),
            layers.Dense(10, activation="softmax"),
        ]
    )
    optimizer = hp.Choice(name="optimizer", values=["rmsprop", "adam"])
    model.compile(
        optimizer=optimizer,
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"],
    )
    return model
```

The function takes `hp` and returns a **compiled model**. Four kinds of range exist: `Int`, `Float`, `Boolean`, `Choice`.

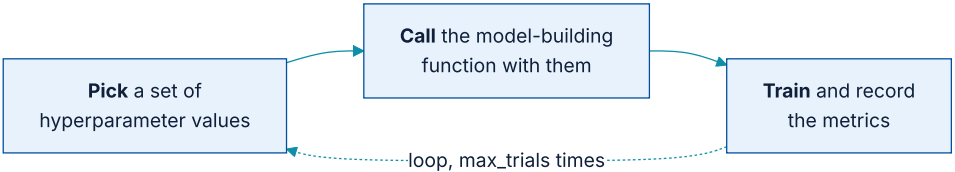
Or subclass HyperModel for something configurable

listing 18.2 PYTHON

```python
import keras_tuner as kt

class SimpleMLP(kt.HyperModel):
    def __init__(self, num_classes):
        self.num_classes = num_classes

    def build(self, hp):
        units = hp.Int(name="units", min_value=16, max_value=64, step=16)
        model = keras.Sequential(
            [
                layers.Dense(units, activation="relu"),
                layers.Dense(self.num_classes, activation="softmax"),
            ]
        )
        model.compile(...)   # exactly as in listing 18.1
        return model

hypermodel = SimpleMLP(num_classes=10)
```

`build()` is the same function as before. The gain is **reuse**: one hypermodel, any number of classes.

The tuner is a for loop with a strategy



Three built-in tuners: RandomSearch, BayesianOptimization, Hyperband.

BayesianOptimization makes smart predictions about which new values are likely to perform best, given the outcome of previous choices — which is what separates it from random search.

SECTION 18.1.1

Configuring the tuner

PYTHON

```
tuner = kt.BayesianOptimization(
    build_model,
    objective="val_accuracy",
    max_trials=20,
    executions_per_trial=2,
    directory="mnist_kt_test",
    overwrite=True,
)
```

ARGUMENT	WHAT IT CONTROLS
`objective`	The metric to optimize. **Always a validation metric** — the goal of the search is models that generalize.
`max_trials`	How many different configurations to try.
`executions_per_trial`	How many training runs **per configuration**, averaged — this is the answer to the noisy-feedback problem.
`overwrite`	`True` to start fresh; **`False` to resume** a crashed search from the trial logs on disk.

Which direction is better?

PYTHON

```python
objective = kt.Objective(
    name="val_accuracy",
    direction="max",
)

tuner = kt.BayesianOptimization(
    build_model,
    objective=objective,
    ...
)
```

`name` must match what appears in the epoch logs. Get it wrong and the search runs happily while optimizing nothing.

SECTION 18.1.1

Launching the search

PYTHON

```
num_val_samples = 10000
x_train, x_val = x_train[:-num_val_samples], x_train[-num_val_samples:]
y_train, y_val = y_train[:-num_val_samples], y_train[-num_val_samples:]

callbacks = [keras.callbacks.EarlyStopping(monitor="val_loss", patience=5)]

tuner.search(
    x_train, y_train,
    batch_size=128,
    epochs=100,
    validation_data=(x_val, y_val),
    callbacks=callbacks,
    verbose=2,
)
```

Never use your test set as validation data here. You would overfit to it immediately, and your test metrics would stop meaning anything.

SECTION 18.1.1

A large epoch count plus early stopping

The MNIST example above runs in a few minutes. With a typical search space and dataset you will find yourself letting a search run **overnight, or over several days**.

> If the process crashes, restart it with `overwrite=False` and it resumes from the trial logs on disk. **Set the directory somewhere durable before you start a multi-day search.**

Retraining the winners is a separate job

listing 18.3 PYTHON

```python
top_n = 4
best_hps = tuner.get_best_hyperparameters(top_n)
```

When retraining, **fold the validation data back into training** — you are making no more hyperparameter changes, so there is nothing left to evaluate against it. Here that means training on all of MNIST's training split.

But that leaves one parameter unsettled: **how many epochs**. The aggressive `patience` used during the search saved time; it may also have left models underfitted.

First, find the best epoch on the validation set

PYTHON

```python
def get_best_epoch(hp):
    model = build_model(hp)
    callbacks = [
        keras.callbacks.EarlyStopping(
            monitor="val_loss", mode="min", patience=10
        )
    ]
    history = model.fit(
        x_train, y_train,
        validation_data=(x_val, y_val),
        epochs=100, batch_size=128, callbacks=callbacks,
    )
    val_loss_per_epoch = history.history["val_loss"]
    best_epoch = val_loss_per_epoch.index(min(val_loss_per_epoch)) + 1
    return best_epoch
```

The high patience is affordable now: this runs a handful of times, not once per trial.

Then retrain on everything, for 20% longer

PYTHON

```python
def get_best_trained_model(hp):
    best_epoch = get_best_epoch(hp)
    model = build_model(hp)
    model.fit(
        x_train_full, y_train_full,
        batch_size=128, epochs=int(best_epoch * 1.2),
    )
    return model

best_models = []
for hp in best_hps:
    model = get_best_trained_model(hp)
    model.evaluate(x_test, y_test)
    best_models.append(model)
```

The `* 1.2` accounts for the extra data: **more samples per epoch means the same number of epochs is a longer run**, but also that the model can absorb a little more before it starts overfitting.

Two things to remember about tuning at scale

```python
best_models = tuner.get_best_models(top_n)
```

And the warning: **validation-set overfitting**. Because you are updating hyperparameters using a signal computed on validation data, you are effectively **training them on the validation data**, and they will quickly overfit to it. Always keep this in mind.

Tuning is automation, not magic

But search spaces grow **combinatorially** with the number of choices, so turning everything into a hyperparameter is far too expensive. You still need to handpick configurations with the potential to yield good metrics.

> The gain is that your decisions **graduate**: from micro-decisions (*how many units in this layer?*) to higher-level ones (*should I use residual connections throughout?*). And higher-level decisions **generalize across tasks and datasets**.

SECTION 18.1.1

Premade search spaces

kt.applications.HyperXception
A tunable version of the Keras Applications Xception model.

kt.applications.HyperResNet
The same for ResNet. **Add data, run the search, get a pretty good model.**

SECTION 18.1.2

Ensembling, and the blind men

The assumption: different well-performing models trained independently are likely to be good **for different reasons**. Each looks at slightly different aspects of the data, getting part of the truth but not all of it.

> *Blind men meeting an elephant: one touches the trunk — "it's like a snake"; another a leg — "like a pillar". Each has part of the truth.* ***Interviewed together, they can tell a fairly accurate story.***
>
> Section 18.1.2

SECTION 18.1.2

Averaging, and then averaging better

```python
preds_a = model_a.predict(x_val)
preds_b = model_b.predict(x_val)
preds_c = model_c.predict(x_val)
preds_d = model_d.predict(x_val)

final_preds = 0.25 * (preds_a + preds_b + preds_c + preds_d)
```

This only works if the classifiers are **more or less equally good**. If one is significantly worse, the result may be worse than the best single model.

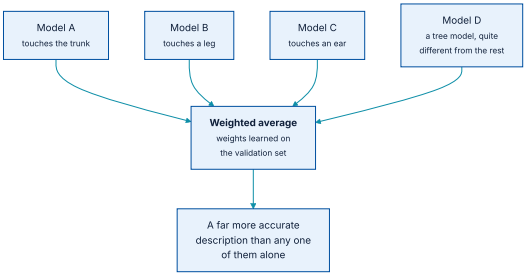
Weighted average, weights learned on validation data

```python
final_preds = (
    0.5 * preds_a
    + 0.25 * preds_b
    + 0.1 * preds_c
    + 0.15 * preds_d
)
```

Better classifiers get a higher weight. To find a good set, use **random search** or a simple optimization algorithm such as **Nelder-Mead** — over the validation data.

> There are many variants — averaging an exponential of the predictions, for instance. In general a **simple weighted average with weights optimized on validation data is a very strong baseline**.

Diversity is strength



If all the blind men touched only the trunk, they would agree that elephants are like snakes — and stay wrong together.

In machine learning terms: **if all your models are biased the same way, the ensemble keeps that bias.** If they are biased differently, the biases cancel and the ensemble is more robust.

As good as possible, while as different as possible

Worth doing

Very different **architectures**, or even different **brands** of machine learning — tree-based methods alongside deep networks.

Largely not worth doing

The same network trained several times from different random initializations. **Low diversity, tiny improvement.**

In 2014 Chollet and Andrei Kolev took **fourth place** in Kaggle's Higgs Boson challenge with an ensemble of tree models and deep networks. One member — a regularized greedy forest — had a **significantly worse score** than the others and was given a small weight.

> It improved the ensemble **by a large factor**, precisely because it was so different: it carried information no other model had. **It is not about how good your best model is; it is about the diversity of your candidates.**

02

Scaling up with multiple devices

Faster training directly improves the quality of your solutions.

SECTION 18.2 · FIGURE 18.1

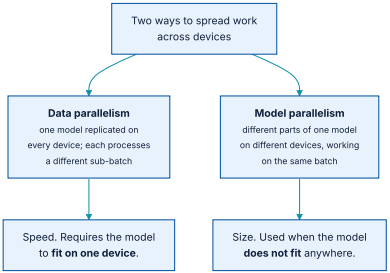
The loop of progress, and its next bottleneck

As your Keras fluency grows, coding up an experiment stops being the bottleneck. **The next bottleneck is training speed.**

Fast infrastructure means results back in 10 or 15 minutes, and hence dozens of iterations a day. **Faster training directly improves the quality of your deep learning solutions.**

SECTION 18.2.1

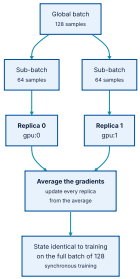
Two kinds of parallelism, for two different problems



Model parallelism is not a way to speed up ordinary models — it is a way to train larger ones.

And you can mix them: split a model across four devices, then replicate that split model twice, for **eight devices total**.

Divide and conquer



Different samples processed in parallel — hence *data* parallelism.

In **inference** you concatenate the sub-batch predictions. In **training** you average the gradients and update every replica from the average, so all replicas hold identical weights at all times. This is **synchronous training**; non-synchronous alternatives exist but are less efficient and no longer used.

Simple, scalable, and it needs the model to fit

> One limitation, and it is decisive at the frontier: the model must **fit on a single device**. It is now common to train foundation models with tens of billions of parameters, which **will not fit on any single GPU**.

That is where model parallelism comes in.

A model too large for one device

listing 18.4 PYTHON

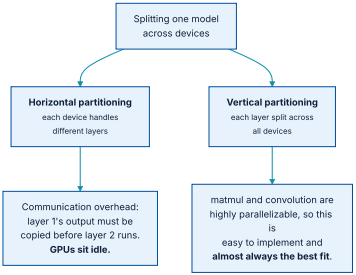
```python
model = keras.Sequential(
    [
        keras.layers.Input(shape=(16000,)),
        keras.layers.Dense(64000, activation="relu"),
        keras.layers.Dense(8000, activation="sigmoid"),
    ]
)
```

~1 billion
parameters in the first Dense

~512 million
parameters in the second

With two small devices, data parallelism is not available — the model does not fit on either. What you can do is **shard** (or *partition*) a single instance across both.

Horizontal and vertical partitioning



Vertical partitioning is almost always the right choice for large models.

Splitting one layer across two devices

PYTHON

```
half_kernel_0 = kernel[:, :32000]
half_bias_0 = bias[:32000]
half_kernel_1 = kernel[:, 32000:]
half_bias_1 = bias[32000:]

with keras.device("gpu:0"):
    half_output_0 = keras.ops.matmul(inputs, half_kernel_0) + half_bias_0

with keras.device("gpu:1"):
    half_output_1 = keras.ops.matmul(inputs, half_kernel_1) + half_bias_1
```

Each device now holds a kernel of shape **(16000, 32000)** instead of (16000, 64000). Nobody writes this by hand at scale — but seeing it once makes the `LayoutMap` API that follows read as ordinary code.

Use JAX

> *We will only cover the JAX backend, as it is the most performant and most scalable of the various Keras backends,* ***by a mile****. If you're doing any kind of large-scale distributed training and you aren't using JAX, you're making a mistake — and wasting your dollars burning way more compute than you actually need.*
>
> Section 18.2.2

For getting hold of the devices, there are two realistic options: acquire two to eight GPUs and mount them in one machine — *it will require a beefy power supply* — or **rent a multi-GPU VM** on Google Cloud, Azure, or AWS with drivers preinstalled. For anyone not training 24/7, the second.

Data parallelism is one line

```python
keras.distribution.set_distribution(keras.distribution.DataParallel())
```

For finer control, list the devices and pass them explicitly:

```python
keras.distribution.list_devices()
# ["gpu:0", "gpu:1", ...]

keras.distribution.set_distribution(
    keras.distribution.DataParallel(["gpu:0", "gpu:1"])
)
```

Note the ordering requirement: **before creating the model**. Setting it afterwards silently does nothing.

N GPUs do not give an N× speedup

~2.0×
with 2 GPUs

~3.8×
with 4 GPUs

~7.3×
with 8 GPUs

Distribution introduces overhead — in particular, **merging the weight deltas** from different devices takes time. The efficiency loss grows with the device count.

> These numbers assume a **global batch size large enough to keep every GPU at full capacity**. If your batch is too small, the local batch will not keep the GPUs busy and the speedup collapses.

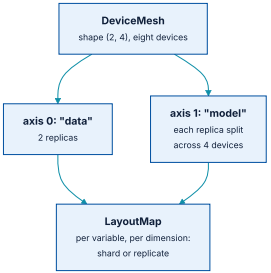
A grid of devices, with named axes

PYTHON

```python
device_mesh = keras.distribution.DeviceMesh(
    shape=(2, 4),
    axis_names=["data", "model"],
)

# or, naming the devices explicitly:
devices = [f"gpu:{i}" for i in range(8)]
device_mesh = keras.distribution.DeviceMesh(
    shape=(2, 4),
    axis_names=["data", "model"],
    devices=devices,
)
```

Two devices along axis 0, four along axis 1: computation is **split across four GPUs**, and **two copies** of that split model each process a different sub-batch. A mesh need not be 2D, but in practice you will only ever see 1D and 2D.

Per variable, per dimension: shard or replicate



Two independent decisions, made per dimension of each variable.

Replicate — every device along that axis sees the same value. **Shard** — a (32, 64) variable becomes four chunks of (32, 16), one per device.

SECTION 18.2.2

Variable paths, and the rule of thumb

PYTHON

```python
for v in model.variables:
    print(v.path)

# sequential/dense/kernel
# sequential/dense/bias
# sequential/dense_1/kernel
# sequential/dense_1/bias
```

For a simple model, the go-to rule is: **shard the last dimension along the `"model"` axis, replicate everything else.**

SECTION 18.2.2

Writing the layout, and turning it on

PYTHON

```python
layout_map = keras.distribution.LayoutMap(device_mesh)
layout_map["sequential/dense/kernel"] = (None, "model")
layout_map["sequential/dense/bias"] = ("model",)
layout_map["sequential/dense_1/kernel"] = (None, "model")
layout_map["sequential/dense_1/bias"] = ("model",)

model_parallel = keras.distribution.ModelParallel(
    layout_map=layout_map,
    batch_dim_name="data",
)
keras.distribution.set_distribution(model_parallel)
```

Once the configuration is set, **no other part of your code changes** — the model definition and the training code are the same, whether you use `fit()` or your own loop.

SECTION 18.2.2

This is the whole of large-scale training

Assuming the right `LayoutMap`, the few snippets you just saw are **enough to distribute any large language model training run** — scaling to as many devices as you have and to arbitrary model sizes.

To check what actually happened, inspect the sharding:

OUTPUT

```
>>> model.layers[0].kernel.value.sharding
NamedSharding(
    mesh=Mesh("data": 2, "model": 4),
    spec=PartitionSpec(None, "model")
)
```

Or visualise it with `jax.debug.visualize_sharding(value.shape, value.sharding)`. **Verify the layout before you pay for a long run** — a silently wrong layout still trains, just slowly.

tf.data performance under distribution

- Always provide data as a `tf.data.Dataset` for best performance. NumPy arrays work too — `fit()` converts them to Datasets anyway.
- Always **prefetch**: call `dataset.prefetch(buffer_size)` before passing to `fit()`.
- If unsure of the buffer size, use `dataset.prefetch(tf.data.AUTOTUNE)` and let it choose.

The pattern is the same one chapter 8 introduced. It matters far more here: a starved input pipeline turns eight expensive GPUs into eight expensive idle GPUs.

SECTION 18.2.3

TPUs: fast enough to be worth the hoops

15×
TPU v2 against an NVIDIA P100

3×
more cost-effective than GPU, on average

TPU v2 is **free in Colab** (Runtime → Change Runtime Type). Google Cloud offers v3 through v5 for serious runs. With the JAX backend, all you need is the same `keras.distribution.set_distribution(...)` call — **before creating your model**.

The data pipeline, and step fusing

GCS read speed becomes the bottleneck

TPUs process batches extremely quickly. If the dataset is small enough, call `dataset.cache()` so it is read from Cloud Storage only once.

Small models underutilize the TPU

Keeping the cores busy can demand batches **upward of 10,000 samples**. And with enormous batches you must raise the learning rate — fewer updates, each more accurate.

Step fusing is the way out: run several training steps per TPU execution, doing more work between round trips to VM memory.

```python
model.compile(..., steps_per_execution=8)
```

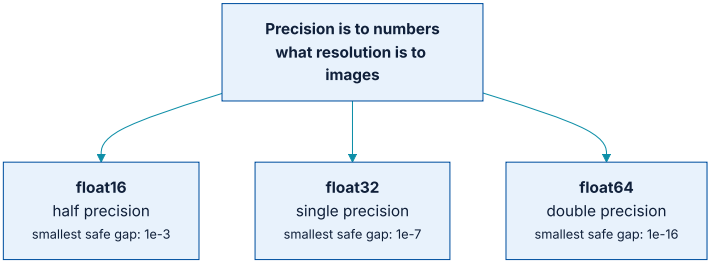
For a small model that underutilizes the TPU, this alone can be a **dramatic** speedup — and it changes nothing else about the training run.

03

Lower-precision computation

Up to 2× faster on almost any model, basically for free.

Precision is to numbers what resolution is to images



How many points depends on how many bits you store the number in.

The practical way to think about it is the **smallest distance between two numbers you can safely process**.

Representable numbers are not uniformly spaced

That falls out of the encoding — sign, mantissa, exponent:

OUTPUT

```
{sign} * (2 ** ({exponent} - 127)) * 1.{mantissa}

Pi in float32:   sign 0 | exponent 10000000 | mantissa 10010010000111111011011
                      1 bit       8 bits             23 bits

value = +1 * (2 ** (128 - 127)) * 1.5707963705062866
value = 3.1415927410125732
```

So the error incurred converting a number to floating point **varies with the value**, and tends to grow with its absolute magnitude.

SECTION 18.3.2

float16 inference: one line, nearly 2×

Modern GPUs and TPUs have hardware that runs 16-bit operations **much faster and with less memory**.

PYTHON

```
import keras

keras.config.set_dtype_policy("float16")
```

Set it **before you define your model**. Expect a nearly **2× speed boost** on `model.predict()` on GPU and TPU.

float16 or bfloat16 — try both

DTYPE	FLOAT16	BFLOAT16
Exponent bits	5	**8**
Mantissa bits	**10**	7
Sign bits	1	1
Character	Finer resolution, narrower range	**Much wider range**, lower resolution

Table 18.1. `bfloat16` covers a far wider range of values at lower resolution over that range, and **works better on some devices, particularly TPUs**.

> Some devices are better optimised for one than the other, so **try both and settle for whichever turns out fastest**. It is a one-line experiment.

SECTION 18.3.3

Training in 16 bits does not work — so use both

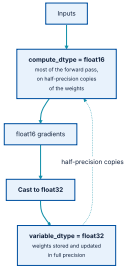
Mixed-precision training is the hybrid: 16-bit computation where precision does not matter, 32-bit where numerical stability does — in particular for gradients and variable updates.

PYTHON

```python
import keras

keras.config.set_dtype_policy("mixed_float16")
```

Most of the speed benefit of 16-bit, without meaningfully impacting model quality.

compute_dtype and variable_dtype



Two dtypes per layer. Both default to float32; mixed precision changes only the first.

Some operations are **numerically unstable in float16** — notably softmax and crossentropy. To opt a specific layer out, pass `dtype="float32"` to its constructor.

SECTION 18.3.4

Loss scaling: multiply the loss so gradients survive

Gradient values are proportional to the loss, so the fix is simple: multiply the loss by a large scalar.

PYTHON

```python
# a fixed factor
optimizer = keras.optimizers.Adam(learning_rate=1e-3, loss_scale_factor=10)

# or let the optimizer work it out
optimizer = keras.optimizers.LossScaleOptimizer(
    keras.optimizers.Adam(learning_rate=1e-3)
)
```

`LossScaleOptimizer` **is usually the better option** — the right scaling value can change over the course of training.

SECTION 18.3.5

Why float8 is not simply the next step down

At float8 that stops being true; you lose too much information. float8 is still usable in *some* computations, but it requires considerable modifications to the forward pass. **You cannot simply set `compute_dtype` to float8 and run.**

Keras has a built-in implementation targeting Transformers, covering only `Dense`, `EinsumDense`, and `Embedding`. It tracks past activation values to rescale activations each step so as to use the full float8 range, and overrides part of the backward pass to do the same for gradients.

float8 can make your model slower

That added machinery has a **computational cost**. If your model is too small or your GPU not powerful enough, the cost exceeds the benefit and you get a **slowdown, not a speedup**.

Model size
Viable only for **very large** models, typically over **5 billion parameters**.

Hardware
Large, recent GPUs such as the **NVIDIA H100**.

float8 is rarely used in practice, except in foundation model training runs. Worth knowing exists; not worth reaching for.

SECTION 18.3.6

int8 quantization: a different trick

matmul is linear, so the final unscaling cancels the initial scaling exactly.

Any loss of accuracy comes **only from the rounding** that happens when casting to int8 — not from the matmul itself.

Doing it by hand, once

PYTHON

```python
from keras import ops

x = ops.array([[0.1, 0.9], [1.2, -0.8]])
kernel = ops.array([[-0.1, -2.2], [1.1, 0.7]])

def abs_max_quantize(value):
    abs_max = ops.max(ops.abs(value), keepdims=True)
    scale = ops.divide(127, abs_max + 1e-7)
    scaled_value = value * scale
    scaled_value = ops.clip(ops.round(scaled_value), -127, 127)
    scaled_value = ops.cast(scaled_value, dtype="int8")
    return scaled_value, scale

int_x, x_scale = abs_max_quantize(x)
int_kernel, kernel_scale = abs_max_quantize(kernel)
```

`+ 1e-7` avoids dividing by zero. Rounding and clipping **before** casting is more accurate than casting directly.

How accurate is it?

PYTHON

```
int_y = ops.matmul(int_x, int_kernel)
y = ops.cast(int_y, dtype="float32") / (x_scale * kernel_scale)
```

OUTPUT

```
>>> y
array([[ 0.9843736,  0.3933239],
       [-1.0151455, -3.1965137]])

>>> ops.matmul(x, kernel)
array([[ 0.98      ,  0.40999997],
       [-1.        , -3.2       ]])
```

Accurate to about three decimal places. For a large matmul this saves a great deal of compute — int8 can be considerably faster than even float16 — and all we added were **fast elementwise ops**: abs, max, clip, cast, divide, multiply.

In practice, one method call

```python
model = ...
model.quantize("int8")
predictions = model.predict(...)
```

`predict()` and `call()` now run partially in int8. **The weights are converted in place**, so this is a one-way operation on that model object — quantize a copy if you still need the float32 version.

CHOOSING BETWEEN THEM

Which lever to pull, and when

TECHNIQUE	WHAT IT BUYS	WHAT IT COSTS
Hyperparameter search	The last few points of accuracy	Hours to days of compute; validation-set overfitting risk
Ensembling	More than any single model can reach	N× inference cost; needs genuinely diverse members
Data parallelism	Up to ~7.3× on 8 GPUs	Model must fit on one device; large batches needed
Model parallelism	Models that fit nowhere else	A LayoutMap to design and verify
Mixed precision	Near 2× on training, nearly free	Loss scaling; unstable ops opted out by hand
int8 quantization	Faster inference than float16	A small, measurable accuracy loss

COMMON FAILURE MODES

Four ways this chapter goes wrong in practice

Tuning against the test set

Passing test data as `validation_data` to `search()`. Every number you report afterwards is contaminated, and nothing warns you.

Ensembling identical models

Same architecture, different seeds. Low diversity, negligible gain, N× the inference bill.

set_distribution after model creation

It silently does nothing. Both `set_distribution` and `set_dtype_policy` must come **before** the model exists.

Mixed precision without loss scaling

Small gradients round to zero in float16 and the model quietly stops learning. Use `LossScaleOptimizer`.

SUMMARY

What has to survive this chapter

1. **Hyperparameters cannot be learned by gradient descent** — the space is discrete, the feedback expensive and noisy.
2. **KerasTuner replaces constants with ranges**; a tuner is a loop with a strategy, and BayesianOptimization is the strategy worth defaulting to.
3. **Tuning is automation, not magic.** You still design the search space, and your decisions graduate from micro to architectural.
4. **Validation-set overfitting is real**: tuning trains hyperparameters on the validation data.

SUMMARY

What has to survive this chapter (2 of 2)

1. **Ensembling works on diversity**, not on individual quality — a worse model of a different kind can lift the whole ensemble.
2. **Data parallelism is for speed, model parallelism is for size.** JAX, DeviceMesh, LayoutMap — and nothing else in your code changes.
3. **N GPUs do not give N× speedup**: about 7.3× on eight, and only if the global batch keeps every device busy.
4. **Mixed precision is close to free**: float16 compute, float32 variables, loss scaling to keep small gradients alive.
5. **float8 is not the next step down** — it needs a rewritten forward pass, 5B+ parameters, and recent hardware to pay for itself.
6. **int8 quantization** scales into [-127, 127], multiplies, and unscales — and `model.quantize("int8")` does it for you.

NOTEBOOK
04_mixed_precision_and_loss_scaling.ipynb

NEXT
Chapter 19 — The future of AI